Chapter 3 - Lists (of nodes)

Lists are present in real life in many different situations, so it's quite natural to try to use a similar structure when making programs on some programming language. Even many programming languages include some form of lists in their repertories.

What do we understand as a list? List is usually a set of elements one after another. Like:

- Car
- Horse
- Plane

Lists usually are indexed, so we can access the elements by their indexing number, and they are insertion ordered. That means that the index of the elements depends on what order they have been inserted or added to the list. It is also possible to insert an element to a list in a certain position referenced by its index.

But one thing should be noted is that when people use the word "list" in normal life, they are referring to a somehow ordered arrangement of elements. But, in programming, what people understand as a list can be implemented in different ways. When, in programming, we talk about lists, we are actually talking about **list of nodes**, that is a very concrete data structure with certain rules. This distinction is

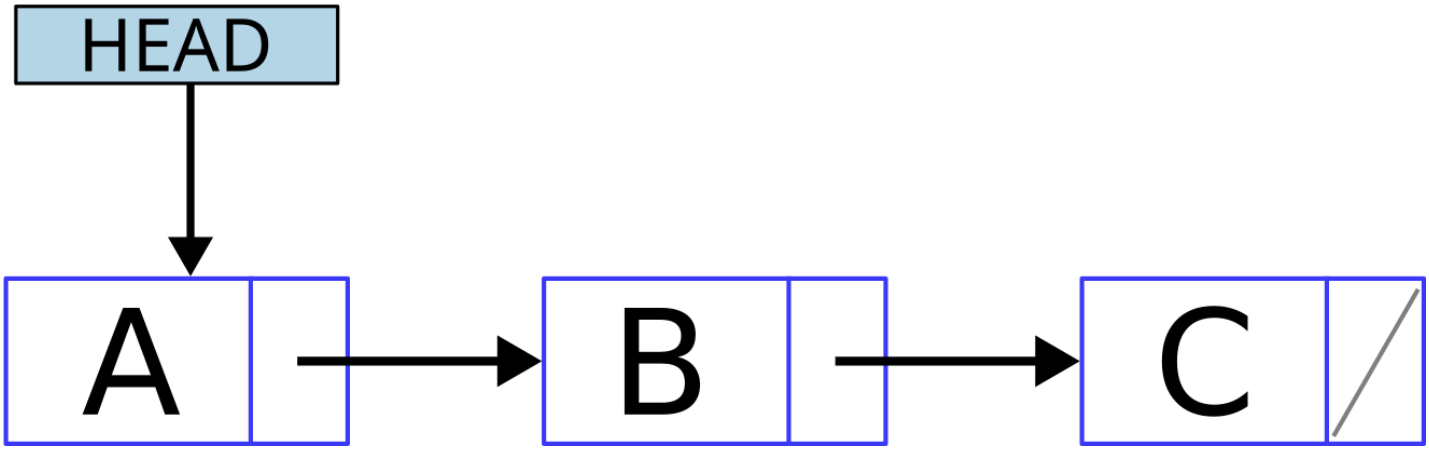
important because there a lot of documentation that confuses the general meaning of a list with its implementation.

For example, Python is one of those programming languages where lists have a main role. Lists in Python are very powerful and widely used. This is in part because Python hides the complex structures that lies in the underneath and offers a quite simple syntax to the user that is very easy to use. But Python lists are actually implemented like dynamic arrays. So, yes, they are lists, but they are lists implemented with arrays.

Anyway, in this chapter we are going to talk about list of nodes, although most of the time just the word "list" will be used.

Singly linked lists

So, in a list (a list of nodes), instead of storing data sequentially, we will use something to store the data, that it is usually called a node. But nodes are independent structures itself, so every node in the list should also have a pointer to the next node to actually work as a list and the last node points to NULL (or None in the case of Python). This type of structure is called a **Singly linked list**.



As the name suggests there are other types of linked lists, but let's not get ahead of ourselves. In general, these kind of structures that use nodes and pointers to somehow link the nodes are called **pointer structures** and lists are not the only way nodes can be linked together. We will see more structures like this in the coming chapters.

Let's see a node in practice:

```
class ListNode():
    def __init__(self, data=None, next=None):
        self.data = data
        self.next = next

    def __repr__(self):
        return f'<ListNode: {self.data}>'
```

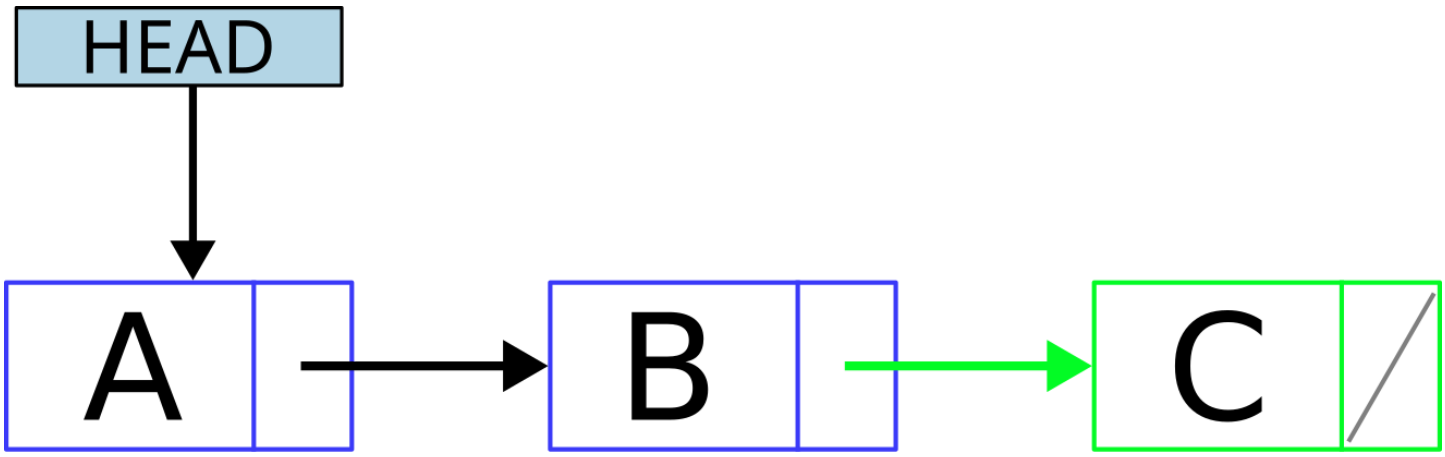
That's it. Just a simple structure to hold some data and a pointer to the next node. We added also a `__repr__` method that can be useful in the Python REPL. Ok, let's try to create a list based in our node class:

```
class SinglyLinkedList():
    def __init__(self):
        self._head = None
```

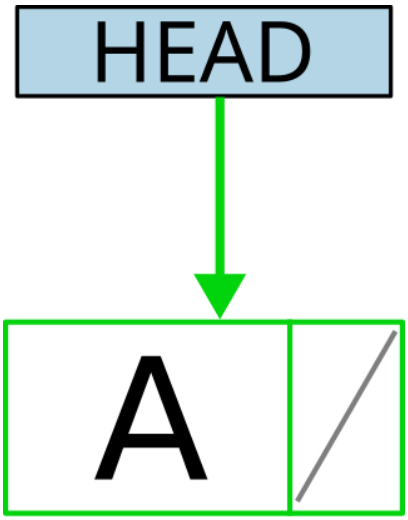
The only thing we need to start is a pointer to the head of our list. We need to know where our list starts. Of course, the list is empty at creation time, so the head is not pointing anywhere.
Let's add a method to add values to the list:

```python
class SinglyLinkedList():
    def __init__(self):
        self._head = None

    def append(self, value):
        """
        Append a value to the end of the list

        Parameters:
        - 'value': The data to append

        Returns: None
        """
        # Create the node with the given value
        node = ListNode(value, next=None)

        # If list is empty just point the header to the new node
        if self._head is None:
            self._head = node
        else:
            # if list is not empty, locate the last element and point it to the new node
            current_node = self._head
            while current_node.next != None:
                current_node = current_node.next
            current_node.next = node
```

The `append` method is not very complicated. Create a new node that points to None, locate the last element, and point it to the new node.



The only extra case it has to take into account is when the list is empty, where the head is what have to be updated.



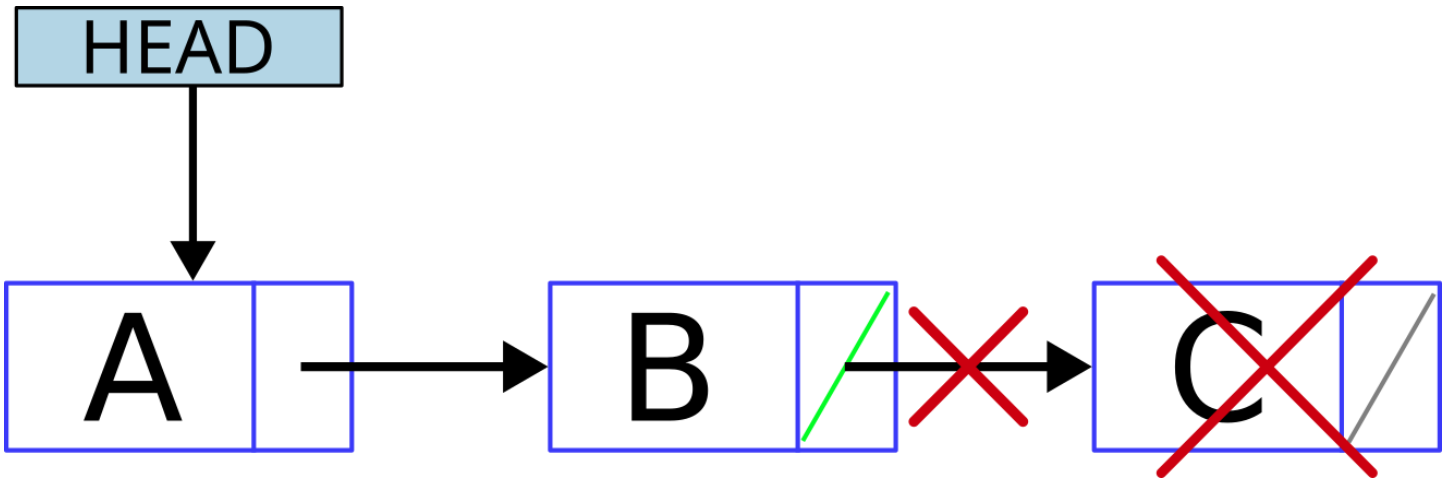
Piece of cake! 😃

Let's also add a `__repr__` method to get some view of the list:

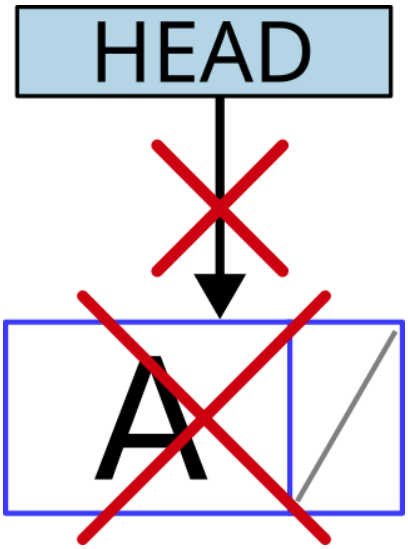
```
    def __repr__(self):
        current_node = self._head
        values = ''
        while current_node:
            values += f', {current_node.data}'
            current_node = current_node.next
        return f'<SinglyLinkedList: [{values.lstrip(", ")}]>'
```

We still can do better, but right let's build thing step by step.

To remove the last element/node, the node before it has also to be located, because its next field has to be updated to `None`.



The only different case to consider is when the list has only one node. In that case, the head has to be updated.



Now let's practice a little bit before moving forward. Turn to the Moodle environment and complete exercise 1 of the 3rd chapter, where you will have to implement a pop method.

Exercise!

Open the Moodle environment and write a method to implement a pop operation

Don't continue until you have finished the exercise!

You will probably have done something like:

```python
    def pop(self):
        """
        Remove a node from the end of the list returning its value.

        - Parameters: None

        - Return: Content of the removed node. If list is empty returns None.
        """
        # If list is empty return None
        if self._head is None:
            return None

        # Find the last node and the one before that
        previous_node = None
        current_node = self._head
        while current_node.next:
            previous_node = current_node
            current_node = current_node.next

        # If node to be deleted is head, update head
        if self._head == current_node:
            self._head = None
        else:
            # If not, update previous node
            previous_node.next = None

        # Finally remove the node and return its value
        value = current_node.data
        del(current_node)
        return value
```

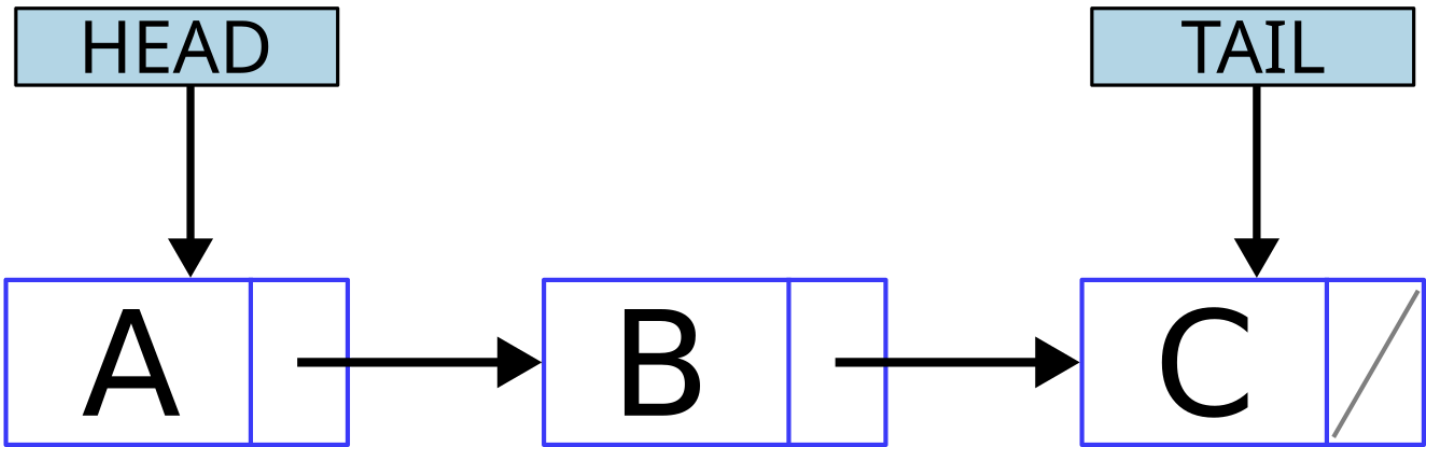
The list works, but is it efficient? Let's see...:

- Creation: $O(1)$. That's good.
- Insertion at the end: $O(n)$. Ok, that's a bummer, but it was clear. We have to traverse the whole list to find the last node to fulfill the task. As this is a common task, it is probably a good idea to add a property that keeps track of the last element.
- Remove from the end: $O(n)$. This one is a little bit trickier, because to complete the task, not only the last node has to be found, but also the one before that. We will solve this one and find a better way to remove from the end a little bit later.
- Get list length: $O(n)$. Right now, the list should be traversed to find out how many elements it contains. This can be solved maintaining a size property separately.

There are more tasks that should be considered, but let's solve some of the problems and let's see after that again.

Singly linked list with tail

So, the idea is to add a size property and a pointer to the last element into the class. As we have seen the start of the list is commonly called 'head'. The end of the list it's called 'tail'.



So now, the list's initialization could be something like:

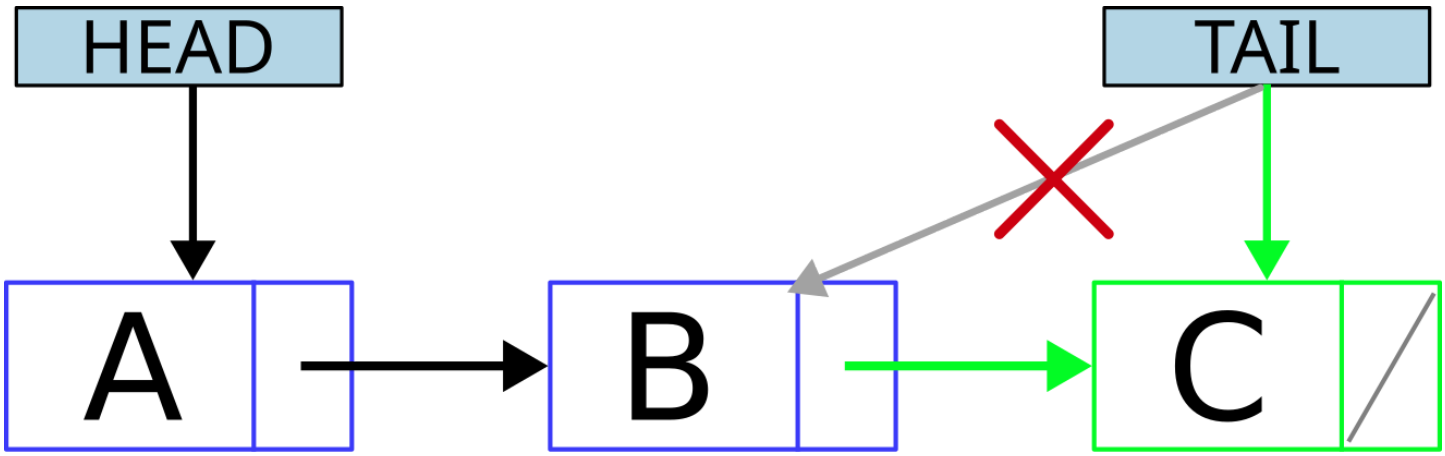
```
    def __init__(self):
        self._head = self._tail = None
        self._size = 0
```

And we can adapt other methods too:

```
    def __repr__(self):
        current_node = self._head
        values = ''
        while current_node:
            values += f', {current_node.data}'
            current_node = current_node.next
        plural = '' if self._size == 1 else 's'
        return f'<SinglyLinkedList ({self._size} element{plural}): [{values.lstrip(", ")}]>'

    def __len__(self):
        return self._size
```

```python
    def append(self, value):
        """
        Append a value to the end of the list

        Parameters:
        - 'value': The data to append

        Returns: None
        """
        # Create the node with the value
        new_node = ListNode(value)

        # If list is empty, point the header and the tail pointers to the new node
        if self._tail is None:      # Or self._head is None
            self._head = self._tail = new_node
        else:
            # if not, update the last node and point it to the new node
            # and also update the tail pointer itself
            self._tail.next = new_node
            self._tail = new_node

        # Update list's size
        self._size += 1
```

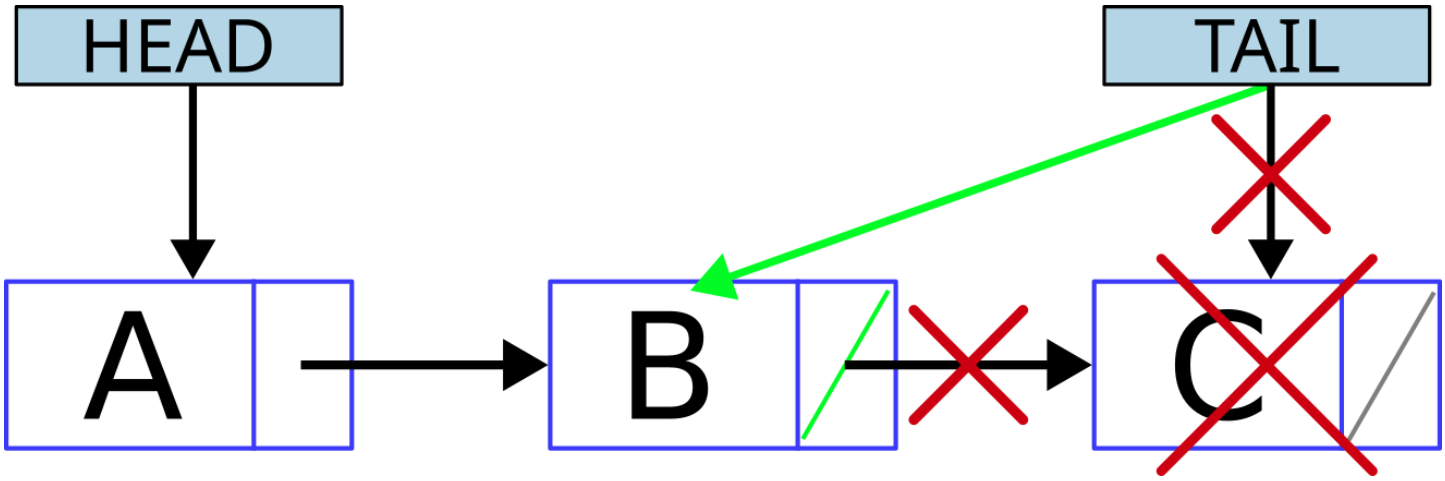
`Append` method is the same as before but also updating the tail pointer.



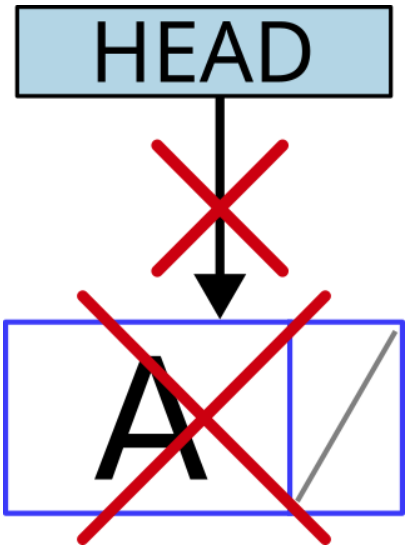
And in the case of the `pop` method, the tail pointer does not affect very much, as the problem is that it also has to find the node before the last. Maybe another pointer to the previous node could be added, but soon we will see there is a better approach.

In any case, let's practice again and open the Moodle environment to complete the second exercise.

In this case we want to implement the pop method, and same as before, apart from deleting the last node, it has to update the pointer of the node before the last one and change the tail pointer to this node.



The special case is when there is only one node, where the head should also be updated.



Exercise!

Open the Moodle environment and write a method to implement a pop operation for a list with tail pointer.

Don't continue until you have finished the exercise!

Your answer should be something like:

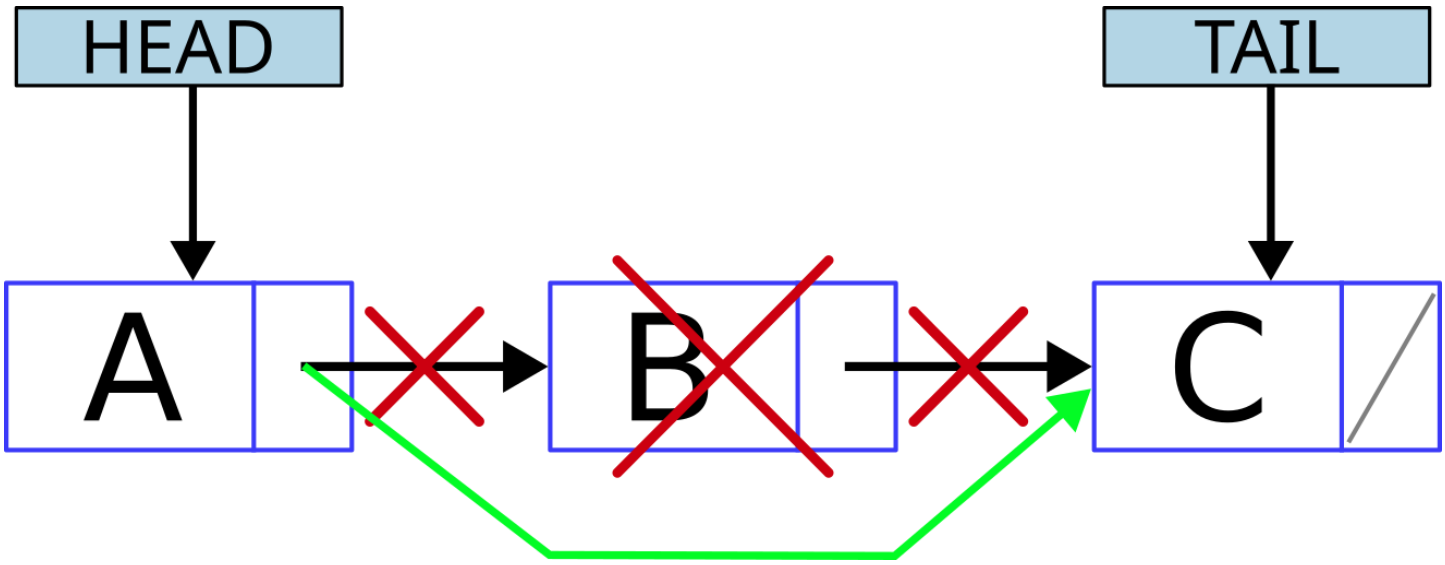
```python
    def pop(self):
        """
        Removes the last node of the list

        Parameters: None

        Returns:
            The content of the removed node. If list is empty, returns None
        """
        # If list is empty return None
        if not self._size:
            return None

        # Locate previous_node (the node just before last node)
        if self._size == 1:
            previous_node = None
        else:
            previous_node = self._head
            for _ in range(self._size-2):
                    previous_node = previous_node.next
        node_to_remove = self._tail

        # If node to remove is the first node, then update head
        if previous_node is None:
            self._head = None
        else:
            # If not, update previous node
            previous_node.next = None

        # Update tail
        self._tail = previous_node

        # Update size, remove node and return its content
        self._size -= 1
        value = node_to_remove.data
        del(node_to_remove)
        return value
```

Note

The `del` statement just removes the identifier it is given, not the node it points to. Python works with a garbage collector, so it is not necessary to remove the node, but I wanted something to represent that the node should be deleted, even if the exact details depend on the programming language of use.
The same statement is going to be used in other parts of this course and the same principle applies.

While you are still warm 😃, you can complete exercise 3 in the Moodle environment, where you will implement a more general remove method. To help you with the task, an `insert` method will be also provided that works as an example.

The `remove` should take into account the same cases as before, but also when the node to remove is in the middle. When this happens, head and tail pointers are not touched, but, as before, the pointer of the previous node should be updated. So, in this case we need to locate the node to be deleted and the nodes before and after it.



Exercise!

Open the Moodle environment and write a method that implements a remove operation.

Don't continue until you have finished the exercise!

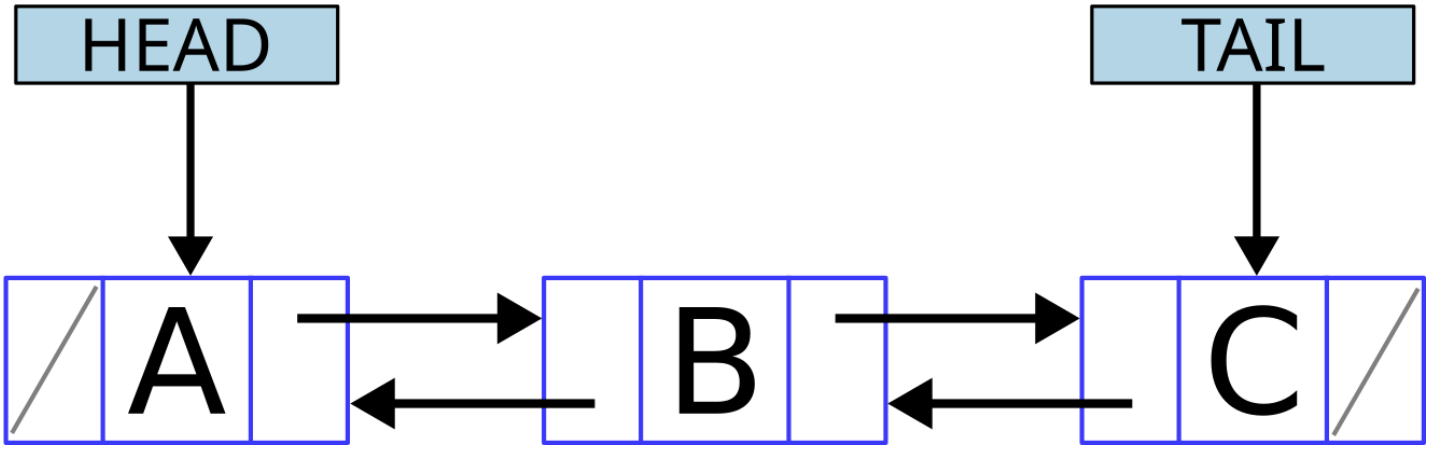
Let's review time complexity for singly linked lists:

- Indexing (finding an element in the list): $O(n)$. Compared to arrays this is worse.
- Add element at front: $O(1)$
- Remove element from the front: $O(1)$
- Add element at end: $O(n)$ without tail and $O(1)$ with tail
- Remove element from the end: $O(n)$ with or without tail, because previous element has to be located
- Add element in the middle: $O(n)$ if we have to locate the element. It would be $O(1)$ if we have a pointer to the node beforehand
- Remove element from the middle: $O(n)$ because previous element has to be located in any case.
- Get size of list: $O(1)$ if size property is used.

Doubly linked lists

Things can be a little bit simpler if we can locate the previous element of a node fast. As we have done before when we need to speed things up, we can just add this information to the list. Every node can have, in addition to the pointer to the next node, a pointer to the previous element. This is what is called **Doubly linked list**.

From any node you can move forward or backwards:



This can simplify also the code overall. Let's review how code looks. First, the ListNode class needs to add the "previous" property:

```python
class ListNode:
    def __init__(self, data=None, next=None, prev=None):
        self.data = data
        self.next = next
        self.prev = prev

    def __repr__(self):
        return f'<ListNode: {self.data}>'
```

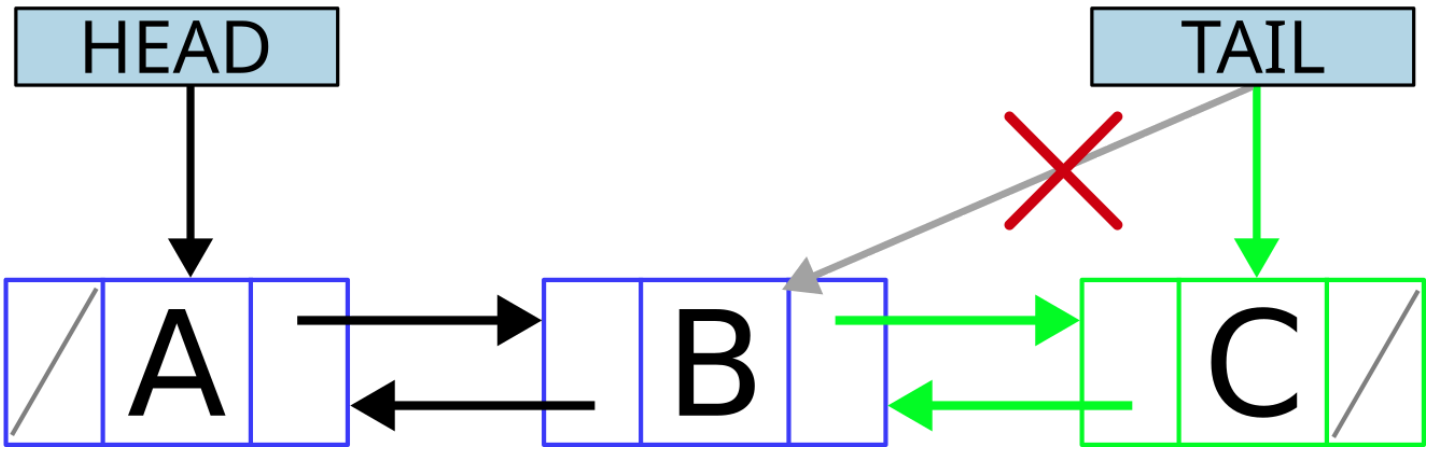
Apart from that everything is the same. We create a new class `DoublyLinkedList` for the new code, but it will be very similar to the singly linked list one.

```python
class DoublyLinkedList:
    def __init__(self):
        self._head = self._tail = None
        self._size = 0

    def __repr__(self):
        current_node = self._head
        values = ''
        while current_node:
            values += f', {current_node.data}'
            current_node = current_node.next
        plural = '' if self._size == 1 else 's'
        return f'<DoublyLinkedList ({self._size} element{plural}): [{values.lstrip(", ")}]>'

    def __len__(self):
        return self._size
```

The `append` method is also almost the same:

```python
    def append(self, value):
        """
        Append a value to the end of the list

        Parameters:
        - 'value': The data to append

        Returns: None
        """
        # Create the node with the value. This is the last node, so next is None,
        # but there can be already some other node in the list, hence the prev value
        new_node = ListNode(value, next=None, prev=self._tail)

        # If list is empty, update head and tail pointers
        if self._head is None:
            self._head = self._tail = new_node
        else:
            # In any other case, update tail node to point to the new element
            # and update tail pointer. The new node already points to its
            # previous element
            self._tail.next = new_node
            self._tail = new_node

        # update size
        self._size += 1
```

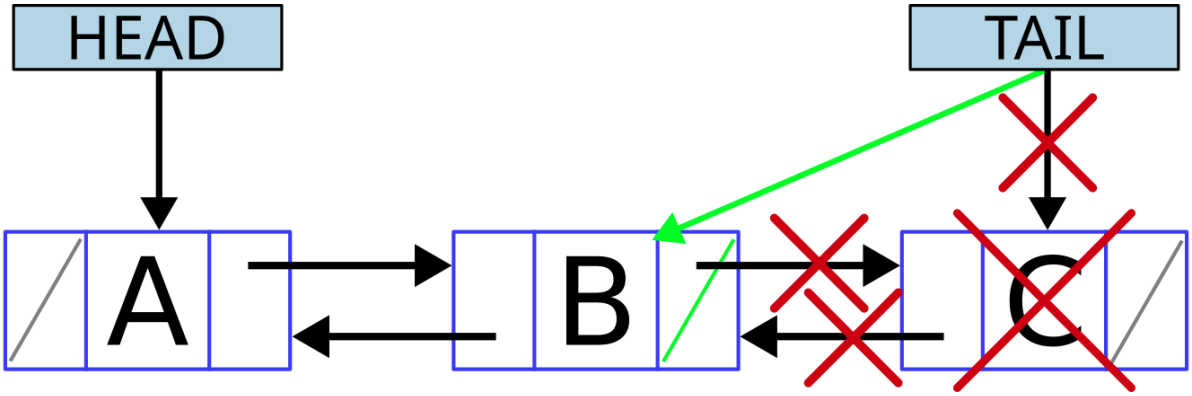
Visually is something like this:



But code also takes into account if the list is empty and head pointer has to be modified.

The `pop` method is also almost the same as the one in the Singly linked list, but with a big difference, it is now of time complexity $O(1)$, as it doesn't need to traverse the list to locate the previous node.

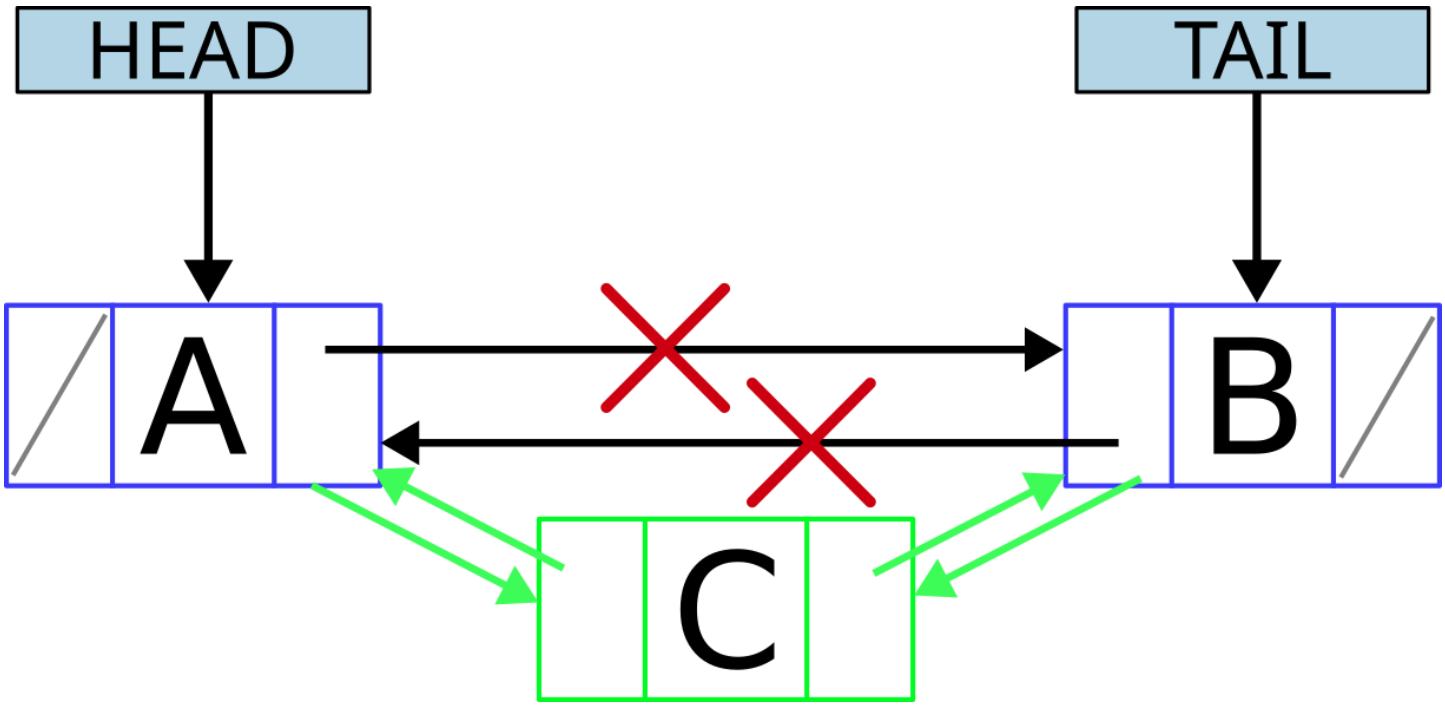
```python
    def pop(self):
        """
        Removes the last node of the list

        Parameters: None

        Returns:
            The content of the removed node. If list is empty, returns None
        """
        # If list is empty, returns None
        if not self._size:
            return None

        # Locate previous_node (the node just before last node)
        node_to_remove = self._tail
        previous_node = node_to_remove.prev

        # If node to remove is first node, then update head pointer
        if node_to_remove == self.head:
            self._head = None
        else:
            # If not, update the pointer of the previous node
            previous_node.next = None   # It is now the last node

        # Update tail pointer
        self._tail = previous_node

        # Update size, remove node and return its content
        self._size -= 1
        value = node_to_remove.data
        del(node_to_remove)
        return value
```



And something similar happens with the `insert` and `remove` methods. The node to be deleted still has to be located and this is of time complexity $O(n)$. That can't be avoided in lists. But once it is located, the insertion or remove can be done in constant time ($O(1)$).

But you can probably implement these two methods on your own. So, once more, you will open the Moodle environment to complete the two final exercises of this chapter.

In the first of them, you will implement the insert method, which inserts a new element/node at a position given as a parameter. The process is quite straight. Locate the nodes that will be before and after the new node and update the necessary pointers. If the header pointer is affected, it should be updated. And the same happens with tail pointer.

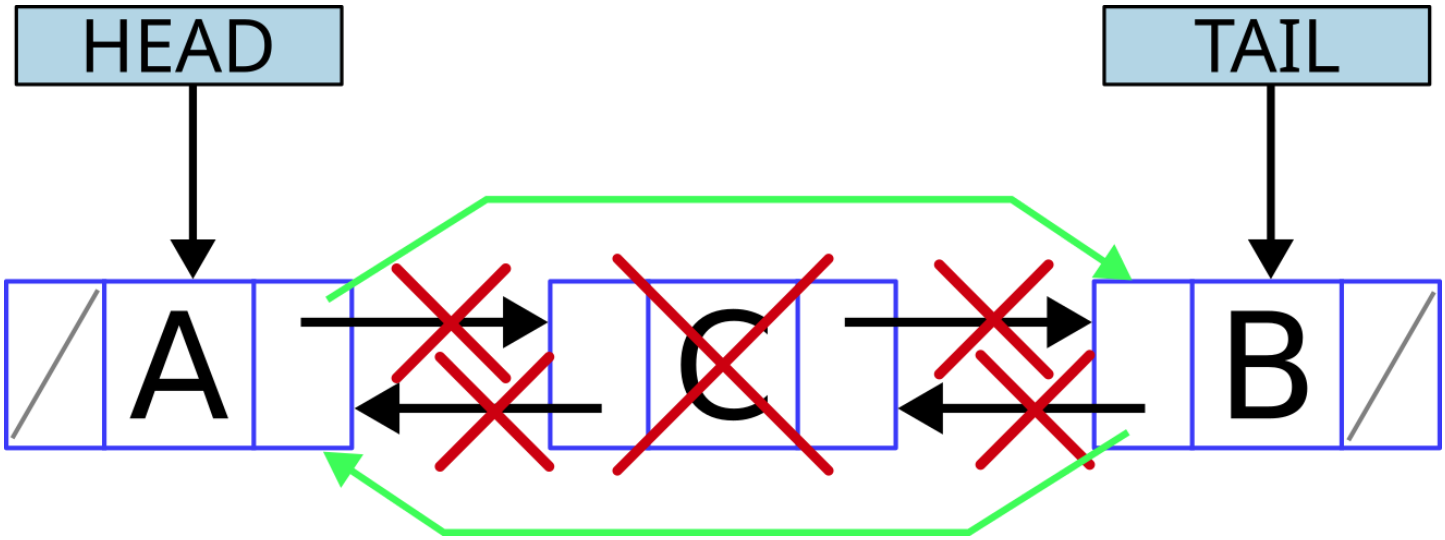


Exercise!

Open the Moodle environment and write a method that implements an insert operation on a doubly linked list.

Don't continue until you have finished the exercise!

In the second of them, you will implement the remove method, which removes an element/node from the position given as a parameter. Same as before, the process is quite straight: Locate the nodes before and after the one to remove and update the necessary pointers. If the header pointer is affected, it should be updated. And the same happens with tail pointer.



Exercise!

Open the Moodle environment and write a method that implements a remove operation on a doubly linked list.

Don't continue until you have finished the exercise!

Let's review time complexity for singly linked lists:

- Indexing (finding an element in the list): $O(n)$. We can't do better in lists.
- Add element at front: $O(1)$
- Remove element at front: $O(1)$
- Add element at end: $O(1)$
- Remove element from the end: $O(1)$
- Add element in the middle: $O(n)$ if we have to locate the element. It would be $O(1)$ if we have a pointer to the node beforehand.
- Remove element from the middle: $O(n)$ if we have to locate the element. It would be $O(1)$ if we have a pointer to the node beforehand.
- Get size of list: $O(1)$

There are still some actions with $O(n)$ complexity, but we also have a bunch of ones with $O(1)$ time complexity, so it seems that this is better data structure if our case uses primarily those actions with $O(1)$. If our use case requires many accesses to random positions of the list, then probably it's better to use arrays or some other data structure.